Rapport de réalisation — version initiale

ISSE-05 • Suivi de flotte de véhicules • 6 novembre 2026

Équipe : quatre noms à compléter. Document à personnaliser après validation locale.

1. Résumé

FleetPulse implémente le suivi d'une flotte de véhicules simulés conformément au sujet ISSE-05. Il associe ingestion événementielle, lakehouse Hudi, calculs Spark, orchestration Airflow et carte Streamlit. Ce rapport décrit le code livré ; les résultats de recette sur PC et les contributions individuelles doivent être ajoutés après exécution.

2. Source et ingestion

Le simulateur Python produit douze positions toutes les deux secondes. Un véhicule sort volontairement de la zone rectangulaire. Les vitesses varient et des périodes d'arrêt alimentent le clustering. Le générateur utilise des coordonnées synthétiques ; il ne rejoue pas GeoLife.

Mosquitto transporte le flux fleet/positions avec QoS 1. La passerelle transmet le payload sans modification au topic Kafka positions. Trois partitions et la clé vehicle_id regroupent les événements d'un véhicule. Kafka est configuré en mode KRaft à un seul broker pour une démonstration locale.

3. Data Lake et Hudi

MinIO fournit un stockage S3 compatible. Bronze archive le payload brut et ses offsets Kafka dans un fichier Parquet par micro-lot. Les données invalides sont isolées en quarantaine. Silver reçoit les positions nettoyées et enrichies des indicateurs de dépassement et de sortie de zone.

Hudi Copy-on-Write utilise event_id et event_ms pour les upserts Silver. Les tables Gold portent les bilans journaliers, la dernière position et les incidents. Les partitions correspondent au jour UTC. Un index GLOBAL_BLOOM permet les mises à jour entre partitions, notamment pour gold_latest.

Chaque table Hudi est transactionnelle indépendamment. Le pipeline complet n'est pas une transaction distribuée. Le checkpoint et les chemins Bronze stables rendent les reprises prévisibles, mais une garantie exactly-once globale n'est pas revendiquée.

4. Traitements Spark

Structured Streaming consomme Kafka et déclenche des micro-lots toutes les vingt secondes. La validation filtre les valeurs incohérentes, puis un upsert conserve une seule clé event_id. Les règles d'alerte sont explicites : vitesse > 80 km/h et position hors rectangle autorisé.

Le batch ordonne les positions par véhicule et instant. La distance journalière somme des segments de Haversine ; les intervalles supérieurs à cinq minutes sont ignorés. Le premier segment d'un jour peut partir du dernier point de la veille et est attribué au jour du point d'arrivée. Spark SQL calcule vitesse moyenne, maximum, volume de mesures et mesures en incident.

Les incidents représentent des échantillons GPS, pas des épisodes de conduite regroupés. Une évolution possible consiste à regrouper les mesures contiguës en épisodes et à calculer leur durée.

5. Apprentissage non supervisé

MLlib KMeans regroupe les positions à vitesse <= 5 km/h. Les coordonnées sont converties approximativement en kilomètres dans la zone de Casablanca afin de ne pas mélanger des degrés d'échelles différentes. Le nombre de clusters est au plus trois et au moins deux, selon les points distincts disponibles. La graine vaut 42.

Le modèle est sauvegardé dans MinIO et ses centres sont affichés. Il ne fournit pas une accuracy de classification. Les résultats quantitatifs, la silhouette et la stabilité des centres restent à mesurer. En dessous de deux points distincts, l'entraînement est explicitement omis.

6. Orchestration et restitution

Le DAG refresh_gold s'exécute toutes les deux minutes et appelle un service HTTP interne qui lance spark-submit. Un verrou évite les exécutions concurrentes. Les erreurs déclenchent des tentatives Airflow. Le streaming reste un service continu lancé par Compose.

Le DAG daily_driving_report contrôle la fraîcheur de l'export, les volumes et les distances avant de produire un rapport JSON du jour UTC concerné. Les rapports vides pour des dates sans données sont signalés par vehicles=0. Aucun historique n'est fabriqué.

Streamlit lit des snapshots JSON dérivés des tables Gold Hudi. Il affiche carte, bilan, mesures en incident et centres d'arrêt. Les exports sont relus toutes les trente secondes ; la fréquence réelle des données Gold dépend de la planification et de la durée des calculs.

7. Validation et résultats

Les tests unitaires vérifient une distance géographique connue, les coordonnées identiques, les bornes de qualité, le seuil de vitesse et le géorepérage. La compilation Python et la validation syntaxique Compose ont réussi. La recette locale Spark 3.5.3 a également réussi : validation, dédoublonnage, alertes, distances, agrégats SQL et entraînement KMeans à deux centres. Un test Hudi 0.15.0 sur disque local confirme que deux écritures de la même clé conservent une ligne et mettent à jour la vitesse et son alerte. Le transport S3A/MinIO reste à vérifier dans la recette complète.

La construction Docker complète a été tentée dans le cloud. Elle s'est interrompue avec no space left on device dans le stockage Docker, avant un essai de bout en bout. Cette limite est documentée ;

elle ne permet pas d'affirmer que la plateforme complète a été validée.

À compléter après exécution locale : captures Kafka/MinIO/Airflow/Streamlit ; nombre de lignes Silver ; exemples de distances vérifiées manuellement ; centres KMeans ; durée du batch ; comportement après redémarrage. Consigner les résultats dans VALIDATION.md.

8. Répartition et soutenance

Le PDF impose quatre étudiants, des comptes Git distincts et une présentation équilibrée. La note finale annoncée est composée de 60 % de note collective et 40 % individuelle. Les noms et contributions ne sont pas inventés : compléter EQUIPE.md et montrer le travail réel de chacun.

La soutenance doit suivre une donnée de bout en bout et montrer un DAG réussi, des tables Hudi et la carte. Le diaporama livré contient des notes orales ; il doit être personnalisé avec les résultats et captures obtenus par le groupe.

9. Limites et perspectives

Le système comporte un broker Kafka unique, des comptes de démonstration et aucun mécanisme de haute disponibilité. Le batch relit Silver entièrement et les snapshots sont collectés au driver : ce choix simplifie le démonstrateur mais limite son passage à l'échelle.

Évolutions : agrégats incrémentaux, moteur SQL pour le dashboard, suivi des événements tardifs, rétention Bronze, sécurité MQTT/Kafka/S3, géorepérage par polygones, recalage routier et étude de plusieurs k par silhouette. Les limites de latence et de volume doivent être mesurées, pas supposées.

10. Références

Sujet utilisateur : Projets_BigData_ADIA_IL_ISSE.pdf, guide général et ISSE-05. Documentation technique : spark.apache.org/docs/3.5.3/ ; hudi.apache.org/docs/0.15.0/ ; kafka.apache.org/38/documentation/ ; airflow.apache.org/docs/apache-airflow/2.10.4/ ; docs.streamlit.io ; min.io/docs/minio/ ; mosquitto.org/documentation/. Ces liens sont des références de documentation, pas des résultats expérimentaux.